

Main StudyOS

Rewrite Plan & Timeline — discussion summary

Project Naming

MSO (Main StudyOS) and **SSO** (Sub StudyOS) are the two related but separate projects referenced throughout this document.

MSO refers to the personal academic operating system project (dashboard, timetable, notes, semesters/subjects, goals, Pomodoro, calendar, and the built-in assistant **Meridian**). **SSO** refers to a separate, personal assistant project (JARVIS / KAREN / F.R.I.D.A.Y.) which also exclusively owns the gesture-control feature. SSO is not covered in this document.

Current State of Main StudyOS

The current build is a single ~5,700-line HTML file using vanilla JS and browser localStorage for all data (no real backend/database). It already includes a substantial feature set: semesters and subjects with progress tracking, goals/CGPA tracking, calendar, study streak, Pomodoro timer, notes, to-dos, and a fully built-in voice assistant named **Meridian** (chat panel, mic input, wake word, voice replies, settings).

Known bugs reported:

- Edit option does not work properly / is hard to use
- Notes formatting (bold, color, lines) is unreliable
- Uploaded PDFs and images in Notes do not open

Decision made: fix and edit all of the bugs above, and change the underlying code/language of the app so it is easier to work with and maintain going forward. Uploaded PDFs and images in Notes should open easily as part of this fix. Priority is reaching a stable base/MVP ready to test with real people as soon as possible.

Gesture control (camera-based hand detection) is **not part of MSO**. It belongs exclusively to the SSO project (a separate, personal project) and is out of scope for this plan.

Root Cause Assessment

Notes formatting bug: Likely caused by storing note content as raw HTML strings rather than structured data, which breaks on re-parsing.

PDF/image upload bug: Likely a storage/permissions or URL-reference issue (no real file storage backend is currently used).

Single-file architecture: A ~5,700-line monolithic file with global state makes bugs hard to isolate and will not scale as more features are added.

No real backend: All data lives in localStorage — tied to one browser/device, at risk of loss, and unusable for multi-device testing with real people.

Recommended Storage & Stack Decisions

Database/Storage: Supabase (Postgres + Auth + Storage) — chosen over Firebase because the app's data is deeply relational (Semester → Subject → Topic → Subtopic), which Postgres models cleanly. Supabase also provides real user accounts (fixing multi-device/profile issues) and file storage buckets with signed URLs (fixing the PDF/image bug at its root).

Planned Code Changes (in priority order)

- 1. Move off the single HTML file** — Rebuild as a proper Next.js + React app with separate components (Dashboard, Semesters, Notes, Calendar, Meridian panel, etc.).
- 2. Add TypeScript** — Type safety catches the data-shape mismatches likely responsible for the edit and Notes bugs.
- 3. Replace the hand-rolled note editor with Tiptap** — Store note content as structured JSON in Supabase instead of raw HTML strings — fixes formatting bugs directly.
- 4. Replace hand-written CSS with Tailwind + shadcn/ui** — Matches the original stack plan; reduces custom CSS maintenance.
- 5. Replace manual state + localStorage writes with Zustand + Supabase** — Keeps UI state in sync with the database automatically.
- 6. Swap hand-drawn progress bars/charts for Recharts** — For goals/CGPA progress and future analytics.
- 7. Add Framer Motion for animations** — Consistent animations as the app grows; matches Linear/Vercel-style design inspiration.

Priorities for the rewrite, as stated: effective, minimum bugs, fast, useful, powerful, and well-prototyped. Steps 1–3 above directly serve the first four; steps 4–7 add polish and power on top of a stable base.

Offline Support

The test/MVP build should work offline. Most of the stack already supports this natively, since these tools run entirely client-side and ship bundled with the app:

- Tiptap — runs in-browser, editing works fully offline
- Tailwind CSS — compiled into a static CSS file at build time, no live/external loading
- shadcn/ui — component code lives in the project itself, no external calls
- Zustand — pure in-memory state, no network dependency
- Framer Motion — client-side animation logic only

The only piece of the stack that requires internet is the remote database/storage/auth layer (e.g. Supabase), since that is a cloud service by nature. To make the app fully usable offline, the plan is an offline-first architecture: a local on-device store (such as IndexedDB via Dexie.js) acts as the source of truth for reads/writes, with background sync to the cloud database whenever internet is available, plus a PWA service worker so the app itself loads offline too.

Database/storage choice and the offline-sync approach are linked decisions and will be finalized together in a later discussion.

Full Project Report — Main StudyOS

A personal academic operating system built for use throughout Harshid's B.Tech — not a public product, a tool for his own studies, designed to eventually be tested with a small group of real students.

Vision & Design

- Works like a modern web app, optionally installable as a PWA
- Dark theme, glassmorphism, rounded cards, smooth animations, professional UI
- Fully responsive across mobile and desktop
- Design inspiration: Notion, Linear, Vercel Dashboard

Core Feature Set

- Dashboard, timetable, and daily planner
- Semester/subject management with unlimited, fully editable semesters and subjects (rename, delete, reorder, icon/color)
- Units/topics/subtopics checklists, where checking off a topic auto-updates subject, semester, and overall degree progress
- Notes (rich text/markdown) with PDF/image storage
- Resources (links), Pomodoro timer, analytics, study streak, calendar
- Assignment tracker, revision tracker, exam mode, goals (including CGPA tracking)
- Built-in voice assistant (currently named Meridian) with chat, mic input, wake word, and voice replies

What's Needed Right Now — Honest Priority Take

Based on everything discussed, here is where the project actually stands and what matters most immediately:

- **Fix the core bugs first.** The edit-option bug and Notes formatting/upload bugs are the biggest blocker to testing with real people — nothing else matters if the basics feel broken.
- **Move off the single-file architecture.** The current ~5,700-line HTML file with localStorage is the root cause of most bugs and cannot scale to the full feature list or multi-device testing.
- **Lock in the database/offline approach.** This decision is still open and blocks real progress on Notes storage, multi-device sync, and the offline-first layer — it should be settled next.
- **Keep gesture control fully out of MSO.** It's confirmed as a SSO-only feature; no need to think about it while building the MSO base.
- **Don't rebuild Meridian's advanced behavior yet.** Voice/wake-word/assistant logic should be ported only after the data layer (Supabase or equivalent) and Notes are stable — building AI features on a shaky foundation just moves the bugs, not fixes them.

- **Protect the end-of-month timeline by sequencing, not rushing.** Foundation and bug-fixing (Weeks 1–2) should not be compressed to fit in analytics/animation polish (Framer Motion, Recharts) — those can safely land in early October.

Timeline (target: end of month)

Based on solo work alongside coursework, roughly 2–3 focused hours/day, starting September 14.

Week	Dates	Focus	Key Tasks
Week 1	Sep 14 – 20	Foundation	Set up Next.js + TypeScript + Tailwind + shadcn/ui. Set up Supabase (auth + relational schema for semesters/subjects/topics/notes/goals/calendar). Rebuild the static UI shell as components.
Week 2	Sep 21 – 27	Core features + bug fixes	Migrate Semester/Subject/Topic management to Supabase with auto-progress calculation. Rebuild Notes with Tiptap + Supabase Storage (fixes formatting + PDF/image bugs). Fix the edit-option bug. Port Timetable/Calendar.
Week 3	Sep 28 – early Oct	Assistant + testing	Port Meridian (voice, wake word, commands) onto the new state/DB. Add Pomodoro, streak, goals. Bug bash, then hand off to a small group of real testers.

Note: hitting a stable, testable MVP by end of month is realistic with consistent daily focus. Additional polish (deeper analytics, Framer Motion animation pass, and later gesture control) will likely extend a little into early October — that is expected, since these are meant to layer on after the base is solid rather than compress into the same month.